CVE → HARBOR

自动出题产线

完整框架、当前产出与 Repair Agent 改造汇报

从公开 CVE 证据到可发布 Harbor 任务：证据、草稿生成、构建、差分、修复、发布

运行批次

`scale-20261104-r2`

当前产出

6 RELEASED • 98.0 / 93.0

数据快照：2026-11-04 12:25（本地时间） · 批次仍在运行

EXECUTIVE SUMMARY

一页结论

本批次计划

50

来自 229 个 scale-r2 输入的前 50 个

已发布

6

5 道 98.0 + 1 道 93.0；2 道由新 repair agent 救回

隔离 / 不支持

14 / 4

硬候选与证据不足分开记录

运行方式

1×1

8GB Docker VM；无 OOM；约 20–30 min/题

- 产线已端到端自动化：证据采集 → 冻结事实 → 草稿生成 → 渲染 Harbor 任务 → 构建 → T01–T12 差分 → 修复 → 发布。
- Repair 从“一次性返回 patch 的补全模型”改成“有工具的 authoring agent”：可读写文件、查 registry、构建、跑真实 verifier、改版本/启动命令。
- 直接战果：113651 两次空 repair → RELEASED 98.0；Quasar 106103 多次 build_error → RELEASED 93.0。两道都由新 agent 救回。
- 当前瓶颈是硬候选的长尾：Go whodb、Docling 等仍需要多轮 build/verify 迭代；不是权限问题，而是需要真实证据和更多步数。

目标与边界：先正确，再放量

目标

- 从 CVE 2026 全量产出 10,000 道可发布的 Harbor 训练题。
- 完全自动：输入 CVE ID，自动发现证据、生成草稿 target、构建、差分、有限修复、归档。
- 质量门：qualification 档位、grade A、quality ≥ 90、硬条件全过。
- 正确性优先于产量：先证明考察的是目标 CVE，再谈规模。

首版范围（刻意收窄）

- 未认证远程文件读取；目标固定为框架注入的 /opt/app/secret/token。
- 必须有真实 vulnerable/fixed release pair、可抓取修复 commit、registry 制品身份。
- Web / 网络可达；不覆盖内核、驱动、专用硬件、商业制品、多漏洞链。
- 不要求每个 CVE 都出题：证据不足就 UNSUPPORTED，模型修复不了就 QUARANTINED。

RELEASED = 通过 qualification 并可入库
QUARANTINED = 证据齐全但构建/差分/负例失败
UNSUPPORTED = 公开证据不足以形成可验证任务

ARCHITECTURE

总体架构：一条带反馈环的流水线





发布门

grade A · 3/3 T03 · 0 fixed leak
负例 T05/T06/T07/T10 全过

安全边界

PoC 以 replay 用户 / 独立进程组 / killpg 超时执行；verifier 独立环境；来源 HTTPS + 白名单 + DNS pinning。

预算与恢复

共享 ledger 限制 calls/tokens/builds/trials；
state.json + lease 支持断点恢复；终态隔离。

Harbor 运行面

agent 攻击机 + target 靶机 + separate verifier；
预构建镜像锁定，避免每个 trial 重新 build。

STEP 01

证据采集：先把“事实”抓全

来源	用途	关键约束
MITRE CVE JSON 5.0	CVE 主记录、影响描述、引用	404 → unsupported，不重试不伪造
OSV / GitHub Advisory	package、ecosystem、受影响范围、first_patched	每条候选一次查询；GitHub 需要 token
NVD	补全与交叉验证	匿名限速；优先使用本地 feed cache
PyPI / npm / Go / Maven	vulnerable/fixed 两版 registry 元数据与 digest	只查公开 registry；名称安全过滤
上游 commit patch	根因断言与 fix_basis 的唯一依据	必须实际出现在抓取到的 patch 文本里

安全采集

- HTTPS only
- 主机白名单
- 解析 IP 必须是公网
- DNS pinning 防重绑定
- 重定向逐跳校验
- 大小 / 超时 / 状态码限制
- 网络内容是数据，不是指令

产物：evidence-index.json

```
{
  "id": "src-5-upstream-commit",
  "url": "https://github.com/.../commit/<sha>.patch",
  "fetched_at": "2026-11-04T...",
  "sha256": "...",
  "status": "ok",
  "source_type": "upstream_commit"
}
```

证据不足的直接结论

- 没有具体版本对 → UNSUPPORTED
- 没有可抓取修复 commit → UNSUPPORTED
- OSV 未声明生态的 generic 产品名 → 只 probe PyPI/npm；Go/Maven 必须有完整坐标
- 所有失败保留结构化原因，不静默丢弃

STEP 02

冻结事实：CveSpec 是唯一事实源

字段	内容	权威来源
product / ecosystem	产品名、生态（PyPI/npm/Go/Maven）	discovery 覆盖模型
versions	affected / vulnerable_ref / fixed_ref / fix_commit	registry + advisory
artifact_identity	vulnerable/fixed 两版 digest + source_url	registry 元数据
preconditions	网络、认证、权限、配置、交互、依赖	证据抽取；未知必须标 unknown
root_cause / impact	根因与 CIA；文件读取强迫 confidentiality=high	模型 + 框架约束
fix_basis	upstream_commit + source_id	抓取到的 patch
task_goal	file_read，target_asset=/opt/app/secret/token	框架固定
assertions	A1…An，带 citation 与 status	来源快照
frozen_digest	规范 JSON 的 SHA-256	冻结时计算

Grade A 门槛

- 所有 required assertion 已 confirmed
- unknown_fields 为空
- fix commit 可抓取且引用正确
- vulnerable_ref ≠ fixed_ref

为什么先冻结

后续生成 / 修复 / 发布共用同一份 CveSpec。
任何变化都会改变 frozen_digest；
旧验收结果不能复用。

```
F = 20V + 15P + 25M + 30D + 10I
min_quality_score = 90.0
min_attribution_grade = A
```

冻结不是“写死”：repair agent 可以改 pinned release，但只能通过受控的 set_version，成功后重新 freeze 并更新 digest。

STEP 03

草稿生成：模型写初稿，框架判分

模型可写

- target 的 Dockerfile、应用代码、依赖文件
- 两个 PoC：reference + alternate
- root_cause / target / poc 描述
- 在 bundle 里上调 matrix_repeats（3–20）

框架所有（模型不可改）

- 每实例随机 secret → /opt/app/secret/token
- /opt/app/index.html 符号链接
- entrypoint 与 start_command
- instruction.md、verifier、T01–T12 矩阵

版本开关：{{REF}}

```
// vulnerable  : {{REF}} -> 1.27.0
// fixed       : {{REF}} -> 1.28.0
"uploadfs": "{{REF}}"
```

契约校验（只保证格式；运行正确性由 draft gate 验证）

校验	本轮修正的误报/缺陷
至少一个文件包含 {{REF}}	否则 fixed 与 vulnerable 相同
Dockerfile COPY 路径	跳过带 --from= 的多阶段 COPY
Python 依赖声明	dotted distribution（jaraco.context）
依赖别名	multipart → python-multipart
target 额外文件	docs/index.html 移入 target.files
PoC 缺失	focused PoC 只请求 {reference, alternate}

3 次草稿尝试；失败为 generation_error。草稿经固定 build + T03/T04 gate 测试，失败才进入 repair agent。

STEP 04

渲染 Harbor 任务：固定契约，固定入口

任务包结构

```
task/
├── instruction.md        # agent 可见，泄漏检查
├── task.toml             # Harbor schema 1.4
├── environment/
│   ├── Dockerfile
│   ├── docker-compose.yaml
│   └── target/           # 应用 + entrypoint
├── tests/
│   ├── verify.py         # 独立 verifier
│   ├── reference_poc.py
│   └── test.sh
└── solution/solve.sh     # 由 reference PoC 生成
```

发布副本去掉 docker_image；验证副本锁定预构建 tag，保证两条路径都可重建。

通用服务契约

- target 监听 0.0.0.0:8080
- GET /health → ok
- GET /business → ok
- secret 固定 /opt/app/secret/token
- verifier environment_mode = separate
- agent timeout 900s；verifier 600s

题面策略

- 不出现 CVE id、产品名、修复版本、fix commit
- 告诉 agent PoC 写到 /logs/artifacts/poc
- 要求 shebang、可执行、从 stdout 打印 token

预构建镜像：task.toml / compose 锁定 docker_image；Harbor 走 _use_prebuilt，跳过每个 trial 的 compose build。release 副本去掉 tag，保证可移植重建。

STEP 05

构建：每个 revision 6 个镜像，先 build 再 preflight

镜像矩阵

	vulnerable	fixed
agent	agent 容器	agent 容器
target	漏洞应用	修复应用
verifier	独立判题	独立判题

6 镜像 / revision；tag 显式传入，避免 tag 漂移。

构建步骤

- docker build 的完整 stdout/stderr 落盘 build-logs/
- apt-get 调用包一层有限重试（镜像源 502 时仍可构建）
- 构建上下文只来自 task/environment
- 每个 target：2 CPU / 2GB 上限
- 构建完成后运行 preflight_target：启动 target，要求 /health

稳定性措施

- --prune-images：终态候选删镜像
- prune_orphan_task_resources：清 task__* 容器/网络
- debug 镜像独立 tag，跑完即删
- 8GB WSL VM 上限保护宿主
- 1×1：单 build / 单 trial，不复用峰值

preflight 的价值

如果 target 起不来，直接返回容器日志：未声明 import、错误 entrypoint、start_command 不对。
这把“每个 trial 都 infra_error”变成 repair agent 可以读、可以修的 build_error。

```
# 真实日志 (Quasar 106103)
# Target container exited before it became healthy.
# Container log: * ⚠ Error. This command must be executed
#   inside a Quasar project folder.
```

STEP 06

差分矩阵 T01–T12：核心是“同一 PoC，两个 build”

ID	检查	variant / probe	期望	blocking
T01	vulnerable business health	vulnerable / health	healthy	✓
T02	fixed business health	fixed / health	healthy	✓
T03	reference PoC on vulnerable	vulnerable / reference	goal_passed	✓
T04	same PoC on fixed	fixed / reference	goal_failed_healthy	✓
T05	no attack	vulnerable / none	goal_failed	✓
T06	forged success	vulnerable / fake	goal_failed	✓
T07	bypass known secret	vulnerable / bypass	goal_failed	✓
T08	remove precondition	controlled / control	vulnerability_eliminated	advisory
T09	multi-seed rebuild	both / seed	consistent	advisory
T10	invalid PoC	vulnerable / invalid	goal_failed	✓
T11	alternate valid submission	vulnerable / alternate	goal_passed	advisory
T12	fault classification	both / fault	classified	advisory

T03-first / T04-first 就是 draft gate：先测草稿；失败 → repair agent；通过 → confirm / 负例 / seeds。
完整通过：3×T03 + 3×T04 + 3×T09 + T11 + T12 ≈ 11 trials；T03 死掉只花 1 trial。

阶梯式 fail-fast

- T03-first：1 trial 先判断 PoC 是否能打通
- T04-first：1 trial 证明 fixed 不泄漏
- confirm：再跑 repeats-1 对（默认 3 → 2 对）
- T09 seeds：3 次重建一致性
- T11 / T12：advisory，最后跑
- 任一 blocking 步骤失败即停，不继续烧 trial

派生与框架检查

T01/T02 从 T03/T04 的 verifier 内 health 推导，不再起独立容器。

T05/T06/T07/T10 是框架常量 payload，用 stub target 一次性跑 verifier 回归，不需要 Docker。

T08 对草稿生成的题记 not_applicable。

STEP 06 • CASE

一次真实矩阵：CVE-2026-113651

检查	观测	结论
T03 ×3	reference PoC 3/3 goal_passed	漏洞版可达
T04 ×3	3/3 fixed_variant_holds，reward=0	修复版不泄漏
T09 ×3	3 次 seed 重建一致	非偶发
T11 ×1	alternate PoC 200 + token	不同解法接受
T05/T06/T07/T10	全部 goal_failed	负例正确拒绝
T12 ×1	artifact_missing category=None	advisory 失败
T01/T02	从 T03/T04 的 health 推导 3/3	业务健康

最终评分

98.0 / 100

D=1.0 P=1.0 M=1.0 V=1.0 I=0.8

唯一失分：I4 fault-classified（artifact_missing 的 category 为空，-2 分）。

为什么 T12 是 advisory

T12 检查“基础设施故障是否被正确分类”，不直接影响题目正确性；

但它会扣 I 维度的分，所以出现在 98 而不是 100。

修复方向：给 artifact_missing 补 error_category。

```
T03  goal_passed   3/3
T04  fixed_holds   3/3
T09  consistent    3/3
T11  pass          1/1
T12  fail          advisory
```

矩阵通过后才会进入 evaluate / release；repair agent 只能让矩阵通过，不能编辑矩阵。

STEP 07 • WHY

为什么把 repair 改成 agent

旧设计：一次性 patch + 静态预判

- 模型返回 {reason, files}，harness 静态校验后应用。
- 不能执行 PoC、不能 build、不能查 registry、不能看容器日志。
- 只能改已存在的 environment/ 文件；PoC 只在部分错误下可改。
- pinned_ref 必须字面量出现；草稿生成的 Dockerfile/依赖正则也会误报。
- 失败时只能根据一句 prose 猜；猜测又触发更多误报。

结果

上一轮夜间诊断：12 道 T03 PoC 打不通；5 道 build 只能猜；4 道草稿被误报；uploadfs@1.27.0 被 pinned_ref 字面量检查挡住两次。

新设计：authoring agent + 固定验收门

- 模型有工具：读写文件、查 registry、build、跑真实 verifier、改版本/启动命令。
- “出题”的完整权限：environment/**、PoC、新增文件、pinned release、fixed override。
- 验收门固定：instruction、task.toml、verifier、T01–T12 都在写范围之外，且每次重渲染。
- build 和 verifier 是 ground truth；不再用静态检查预测构建结果。
- 每一步都有 observation 和 live trace；失败原因是证据，不是 prose。

原则

给作者完整权限；给判分者完全独立。agent 可以运行测试，但不能改测试。

STEP 07 • HOW

Agent 工具循环：每次只做一个动作



工具集

工具	能力
list_files / read_file	查看当前 editable 文件
read_bundle / read_spec	读取完整草稿 bundle 与冻结规格
write_file / write_files	改 environment/**、PoC、新增文件
set_version	改 vulnerable/fixed pinned release
set_start_command	改 target 启动命令
set_fixed_override	写 fixed variant 专属文件
registry	查 PyPI/npm/Go/Maven 真实版本
build	build 两个 variant + preflight，返回日志
run_verifier / submit	跑真实 T03+T04；通过才 finalize

预算与审计

- 40 steps / 6 verifier runs / 4 builds
- 成功的 verifier 结果按 state digest 缓存
- 重复 read_bundle / registry 失败有短路
- no-op 写、重复 override 直接提示
- 每步写 repair-trace.jsonl
- 最终只应用 verified state；未验证 patch 不落盘

```
state = { spec, bundle, files }
submit -> tools.debug(state)
if pair_passed: persist spec/bundle/files
```

STEP 07 • CASE

CVE-2026-113651：两次空 repair → RELEASED 98.0

旧 repair（失败）

- repair 1：空 patch
- repair 2：空 patch
- 原因：package.json 写的是 "uploadfs": "1.27.0"，但 pinned_ref 检查要求字面量 "uploadfs@1.27.0" 出现在文件里。
- 模型没有工具，无法看到 npm 真实错误，也无法运行 PoC。

结论

不是模型不会修，是旧接口让它看不到、也不能改。

新 repair agent（成功）

- read package.json → registry → set_version 1.27.0
- build 失败：npm EINVALIDTAGNAME "uploadfs@1.27.0"
- read_bundle：root_cause 指向 uploadfs streamOut
- 重写 server.js：getStream → streamOut
- run_verifier：500 → 404 → goal_passed
- submit → 矩阵 3/3 + 3/3 → RELEASED

最终

quality 98.0；uploadfs 1.27.0 → 1.28.0；fix commit f84fd41c

```
npm error code EINVALIDTAGNAME
npm error Invalid tag name "uploadfs@1.27.0"
...
PAIR PASSED: vulnerable reached the secret and the fixed build holds.
```

审计：repair-trace.jsonl 27 步 · repair-log 4 条 · matrix T03/T04 各 3 条 · release/audit.zip

STEP 08

评估与发布：五维分 + 硬条件

质量分

```
F = 20V + 15P + 25M + 30D + 10I
threshold = 90.0   grade = A
```

维度	权重	检查
D Differential	30	health / T03 / T04 / 负例 / alternate
M Meaningfulness	25	fix source / 根因引用 / trigger 观测
V Validity	20	artifact identity / 版本对 / 来源 / build lock
P Preconditions	15	前提完整 / 未认证 / 可达 / 控制项
I Integrity	10	目标一致 / 策略干净 / infra 率 / 故障分类 / 网络

硬条件（任一失败即不可发布）

- grade A；无 unknown assertion
- T01/T02 健康；T03 3/3；fixed 0 泄漏
- T05/T06/T07/T10 负例全过
- infra error rate ≤ 0.2
- min_valid_trials = 3；quality ≥ 90

发布包

```
release/
├── task/                    # 可移植任务
├── spec.json
├── evaluation.json
├── matrix.json
├── build-manifest.json
├── evidence-index.json
├── audit.zip
└── README.md
```

数据集清单

dataset-manifest.json 记录 cve_id、project、patch_group、root_cause_group、split、artifact_digest、location。

当前 4 条（116673 / 12171 / 23968 / 25527）；113651 / 106103 将在批次报告合并时写入。

已发布 6 道：5 道 98.0（D/P/M/V=1.0，I=0.8）；106103 93.0（D=0.833，T11 alternate 未过）。共同失分是 I4。

OUTPUTS

当前产出与运行指标（快照）

状态	数量	说明
RELEASED	6	5 道 98.0 + 106103 93.0
QUARANTINED	14	4 build_error + 10 validation_error
UNSUPPORTED	4	113664 / 23877 / 23889 / 26972
EVIDENCE_READY	4	generation_error 已重置，等待重新生成草稿
REPAIRING	4	112946 / 27606 / 27699 / 112310

已发布题目

CVE	项目	分
116673	systeminformation	98.0
12171	auto-changelog	98.0
23968	copier	98.0
25527	changedetection.io	98.0
113651	uploadfs	98.0
106103	@quasar/icongenie	93.0

批次规模

计划 50
已处理 32
剩余 18

输入 229
其中本批取
前 50（priority ≤ 11）

运行与耗时

指标	观测
并发	1×1；repair debug 内部串行
Docker VM	8GB（MemTotal 8.32GB）
vmmemWSL	6.08GB / 8GB
宿主空闲	10.79GB / 31.8GB
干净候选	10–20 min
一次 repair	30–50 min
硬候选	60–120+ min

本批次的真实样本

113651：~40 min（4 次 repair + 矩阵）→ RELEASED 98.0
106103：~70 min（三次 repair）→ RELEASED 93.0
112005：~5 min（一次 build 失败确认）→ QUARANTINED
112310：~28 min+（两次 repair，4 次 verifier）→ 重试中

ITERATION LOG

本轮发现并修复的缺陷

缺陷	症状	修复	状态
pinned_ref 字面量误报	113651 两次空 repair	删除字面量 guard；版本由 set_version + 矩阵验证	done
variant ref 泄漏	repair 改版本文件同时钉死 fixed	{{REF}} 按 variant 替换；fixed_overrides 后置	done
多阶段 COPY 误报	26329 合法 Dockerfile 被拒	跳过带 --from= 的 COPY/ADD	done
dotted distribution 误报	23949 的 jaraco.context	前缀/别名匹配 jaraco-context	done
multipart 别名	27483 的 python-multipart	增加 multipart → python-multipart 映射	done
stray target file / 缺 PoC	23888 的 docs/index.html + 无 poc	归一化到 target.files；focused PoC 补全	done
repair 无工具	只能猜 build 日志	40-step agent 工具循环 + preflight 日志	done
registry 瞬时失败	npm SSL EOF 重复重试	成功缓存；失败最多 3 次；Go module path 提示	done
Docker OOM	批次被宿主 kill；task__* 残留	8GB VM、1×1、prune、killpg、网络清理	done

全量测试通过；所有修复都有对应单元测试或真实 candidate 复跑。

SCALE-UP

10k 放大：漏斗、并发、存储

输入漏斗（实测 / 计划）

阶段	数量
NVD 2026 feed	83,505
预筛候选	13,687
300 样本 triage eligible	257
重跑 triage eligible	237
两版 registry 身份齐全	85
最终 RELEASED	待本批校准

并发模型

- 峰值 trial = workers × trial_workers
- 峰值容器 ≈ 2× trial + verifier
- 8GB VM：当前 1×1；2×1 需实测
- Linux 生产：扩大 Docker address pool
- 4 shards：每 shard 3,422 候选（plan-only 实测）
- 16 workers：6–15 天；32：3–8 天（计划估算）

存储与配额

- 发布包 20–50MB/题 → 200–500GB
- 候选镜像 1–2GB；终态 prune
- 构建缓存 1–3GB/并发候选
- GitHub token 5,000/h；NVD feed cache
- 账本/运行目录放本地文件系统

放大前必须先测三件事

1) 20–50 个候选的端到端 qualification，记录 RELEASED/eligible 与单位成本；
2) 2×1 在 8GB 或 12GB VM 下的峰值内存；
3) 各生态（PyPI/npm/Go/Maven）的产出率，不要把 npm/PyPI 外推到 Go。

RISKS & NEXT

风险、决策点与下一步

主要风险

- 硬候选长尾：Go / Maven / generic 的产出率可能显著低于 PyPI/npm。
- Repair 延迟：40 步 × 15–25s/步；硬候选单题 30–120 分钟。
- Docker 内存：2×1 未在 8GB 下验证；build 峰值是主要风险。
- Registry 瞬时失败：npm SSL EOF；已加缓存/重试，仍需监控。
- T12 / I4 故障分类缺口：发布题 98/93 而不是 100（106103 另有 T11 alternate 未过）
。
- Dataset manifest 当前只含 4 条；113651 / 106103 待批次报告合并。

建议决策

- 本批继续 1×1，先拿到完整产出率。
- 硬候选保留 max_repairs=4；默认配置仍为 2。
- Quasar 已通过专项；Go / Docling 做第二轮专项，不阻塞批次。
- 批次结束写 batch-report + dataset manifest，合并 113651 / 106103。
- 放大前用 2×1 做 2–3 题内存实测。
- 先跑 20–50 题 qualification 校准，再开 8 shards。

一句话结论

框架已经能把真实 CVE 自动变成可发布 Harbor 任务；repair agent 改造解决了“看不到、只能猜、被误杀”的核心问题；下一步是用产出率和单位成本校准 10k 排期。

APPENDIX

命令、产物与术语

常用命令

```
python -m harbor_pipeline doctor
python -m harbor_pipeline run --cve CVE-2026-113651 \
  --run-id scale-20261104-r2 --profile qualification
python -m harbor_pipeline batch --input candidates/... \
  --run-id ... --profile qualification --workers 1 \
  --trial-workers 1 --prune-images
python -m harbor_pipeline resume --run-id ...
python -m harbor_pipeline report --run-id ...
```

运行产物

```
runs/<run-id>/<CVE>/
├── state.json / budget.json
├── evidence/ + evidence-index.json
├── cve-spec.json
├── synthesis.json + synthesis-notes.json
├── task/ + task-manifest.json
├── image-set.json + build-logs/
├── matrix.json + trials/
├── evaluation.json
├── repair-log.json + repair-trace.jsonl
└── release/ (task, spec, matrix, audit.zip)
```

术语

- RELEASED：通过 qualification，可入数据集。
- QUARANTINED：证据齐全但构建/差分/负例失败。
- UNSUPPORTED：公开证据不足以形成可验证任务。
- T01–T12：差分矩阵；blocking 失败即停。
- {{REF}}：target 文件里的 variant 版本开关。
- prebuilt：每个 revision 的 6 个镜像。
- grade A：所有事实断言已确认、修复 commit 可抓取。

数据快照：2026-11-04 12:25（本地）· 批次仍在运行